

```
ts
await supabase.auth.signUp({
  email, password,
  options: {
    emailRedirectTo: 'regalmeet://auth-callback',
  }
})
```

```
data: { display_name: 'Jane Doe' }  
}  
});
```

A profiles row is auto-created via the handle_new_user trigger.

2.2 Sign in

```
ts  
await supabase.auth.signInWithPassword({ email, password });
```

2.3 Sign out

```
ts  
await supabase.auth.signOut();
```

2.4 Password reset

```
ts  
await supabase.auth.resetPasswordForEmail(email, {  
  redirectTo: 'regalmeet://reset-password'  
});  
// On the reset screen:  
await supabase.auth.updateUser({ password: newPassword });
```

2.5 Session lifecycle

Always subscribe before calling getSession:

```
ts  
supabase.auth.onAuthStateChange((event, session) => { /* ... */ });  
const { data: { session } } = await supabase.auth.getSession();
```

2.6 Update email / password

```
ts  
await supabase.auth.updateUser({ email });  
await supabase.auth.updateUser({ password });
```

3. User Profile

Table: profiles

Column	Type	Notes
id	uuid	PK; equals auth.users.id
display_name	text	nullable
avatar_url	text	nullable; public URL inside meeting-files bucket
created_at	timestampz	default now()
updated_at	timestampz	default now()

RLS: a user can SELECT, INSERT, UPDATE only their own row (auth.uid() = id).

3.1 Read own profile

```
ts  
const { data, error } = await supabase  
  .from('profiles')  
  .select('id, display_name, avatar_url, created_at, updated_at')  
  .eq('id', userId)  
  .single();
```

3.2 Update profile

```
ts  
await supabase  
  .from('profiles')
```

```
.update({ display_name, avatar_url })
.eq('id', userId);
```

3.3 Upload avatar

```
ts
const path = avatars/${userId}.jpg;
await supabase.storage.from('meeting-files').upload(path, fileBlob, { upsert: true });
const { data: { publicUrl } } = supabase.storage.from('meeting-files').getPublicUrl(path);
await supabase.from('profiles').update({ avatar_url: publicUrl }).eq('id', userId);
```

4. User Settings

Table: user_settings — RLS: user manages only own row.

Column	Type	Default
user_id	uuid	—
theme	text ('light'/'dark'/'system')	'system'
language	text	'en'
notifications_enabled	bool	true
push_notifications	bool	true
email_notifications	bool	true
meeting_reminders	bool	true
sound_enabled	bool	true
camera_default_on	bool	true
microphone_default_on	bool	false

```
ts
// Read
const { data } = await supabase.from('user_settings').select('*').eq('user_id', userId).maybeSingle();

// Upsert
await supabase.from('user_settings').upsert({ user_id: userId, theme: 'dark', camera_default_on: false });
```

5. Meetings

Table: meetings

Column	Type	Notes
id	uuid	PK
meeting_id	text	6-character public code users share
host_id	uuid	the creator (auth.uid)
title	text	
description	text	nullable
is_active	bool	default true
status	text	'active' 'ended' 'cancelled'
created_at / updated_at	timestampz	

RLS rules:

- Anyone authenticated can SELECT rows where is_active = true AND status NOT IN ('ended', 'cancelled') (used for join-validation).
- Hosts can SELECT/UPDATE/DELETE their own meetings.
- Users can INSERT only meetings where host_id = auth.uid().

5.1 Create instant meeting

```
ts
const meetingCode = Math.random().toString(36).slice(2, 8).toUpperCase();
await supabase.from('meetings').insert({
```

```
meeting_id: meetingCode,
host_id: userId,
title: 'Quick meeting'
});
```

5.2 Validate / join

```
ts
const { data: meeting } = await supabase
  .from('meetings')
  .select('id, meeting_id, title, host_id, is_active, status')
  .eq('meeting_id', code)
  .eq('is_active', true)
  .maybeSingle();
if (!meeting) throw new Error('Meeting not found or ended');
```

5.3 End / cancel

```
ts
await supabase.from('meetings').update({ is_active: false, status: 'ended' }).eq('id', meeting.id);
```

6. Scheduled Meetings & Invitations

Table: `scheduled_meetings` — host-only RLS (`host_id = auth.uid()`).

Key fields: `meeting_id`, `title`, `description`, `scheduled_time` (timestamp), `duration_minutes`, `timezone`, `is_recurring`, `recurrence_pattern` ('daily'|'weekly'|'monthly'), `recurrence_end_date`, `meeting_link`, `status`.

```
ts
await supabase.from('scheduled_meetings').insert({
  meeting_id: code,
  host_id: userId,
  title, description,
  scheduled_time: isoString,
  duration_minutes: 60,
  timezone: 'Africa/Lagos',
  is_recurring: false
});
```

Table: `meeting_invitations` — `scheduled_meeting_id`, `invitee_email`, `invitee_name`, `status` ('pending'|'accepted'|'declined').

Edge function: send-meeting-invitation

- URL: POST <https://ytbilvvrqokjqvzbtpd.supabase.co/functions/v1/send-meeting-invitation>
- Headers: Authorization: Bearer <user-access-token>, Content-Type: application/json
- Body:

```
json
{
  "scheduledMeetingId": "uuid",
  "invitees": [{ "email": "a@b.com", "name": "Alice" }]
}
```

Sends formatted invitation emails and inserts `meeting_invitations` rows.

7. Joining a Meeting (Lobby Flow)

Two-step UX:

1. Knock: guest opens `/<code>`, supplies a display name, broadcasts a `lobby:knock`.
2. Host approves: host receives toast! Admit sends `lobby:admit`, Deny sends `lobby:deny`.
3. On admit, guest inserts a `meeting_participants` row and starts WebRTC.

Realtime channel: meeting-lobby-{meeting_id}

```
| Event | Payload (sender) | Sent by |
|---|---|---|
| knock | { guestId, name, ts } | guest |
| admit | { guestId } | host |
| deny  | { guestId, reason? } | host |
```

ts

```
const lobby = supabase.channel(meeting-lobby-${meetingCode});
lobby.on('broadcast', { event: 'admit' }, ({ payload }) => { if (payload.guestId === myId) joinMeeting(); });
lobby.subscribe() => lobby.send({ type: 'broadcast', event: 'knock', payload: { guestId: myId, name } }));
```

8. Meeting Participants

Table: meeting_participants

```
| Column | Type |
|---|---|
| id | uuid |
| meeting_id | uuid (FK by value to meetings.id) |
| user_id | uuid |
| user_name | text |
| is_host | bool |
| is_muted | bool |
| joined_at | timestampz |
| left_at | timestampz nullable |
| country / city / ip_address | text nullable |
```

RLS: user can INSERT rows where user_id = auth.uid(). Hosts can read/update all participants in their meetings. Users can read their own row.

8.1 Join

ts

```
await supabase.from('meeting_participants').insert({
  meeting_id: meeting.id,
  user_id: userId,
  user_name: displayName,
  is_host: meeting.host_id === userId,
  is_muted: !micDefaultOn,
  country, city
});
```

8.2 Leave

ts

```
await supabase.from('meeting_participants').update({ left_at: new Date().toISOString() })
  .eq('meeting_id', meeting.id).eq('user_id', userId);
```

8.3 Host mutes someone

ts

```
await supabase.from('meeting_participants').update({ is_muted: true }).eq('id', participantId);
```

8.4 Realtime presence (recommended live participant list)

Use a presence channel keyed by meeting code:

ts

```
const channel = supabase.channel(meeting-presence-${meetingCode}, { config: { presence: { key: userId } } });
channel.on('presence', { event: 'sync' }, () => {
  const state = channel.presenceState(); // { userId: [{ name, isHost, ... }] }
});
```

```
channel.subscribe(async (status) => {
  if (status === 'SUBSCRIBED') await channel.track({ name, isHost, mutedAt: Date.now() });
});
```

9. WebRTC Signalling

Regal Meeting uses a mesh topology over Supabase Realtime. Each pair of participants negotiates one `RTCPeerConnection`. Recommended `iceServers`:

```
json
[
  { "urls": ["stun:stun.l.google.com:19302"] },
  { "urls": ["stun:stun1.l.google.com:19302"] }
]
```

For deployments >25 participants you should add a TURN server (e.g. Twilio, Coturn) — the mesh does not scale to 300+ video publishers without an SFU. The web app addresses this by publishing to all but only rendering N tiles (see §15).

9.1 Channel: meeting-{meeting_id}

Broadcast events (always include from and to):

```
| Event | Payload |
|---|---|
| peer-join | { from, name } |
| peer-leave | { from } |
| offer | { from, to, sdp } |
| answer | { from, to, sdp } |
| ice | { from, to, candidate } |
```

9.2 Reference flow

```
A subscribes channel! emits peer-join
B (already in) receives peer-join! creates RTCPeerConnection! createOffer! sends offer
A receives offer! setRemote! createAnswer! sends answer{to:B}
Both exchange ice candidates (event: ice)
```

9.3 Optional companion channels

- meeting-hands-{id} — hand-raised { userName, handRaised, timestamp }
- meeting-reactions-{id} — reaction { userName, emoji, ts }
- meeting-chat-{id} — message { userName, text, ts, fileUrl? }
- meeting-pin-{id} — pin { pinnedUserId } (for selecting the active video frame)

10. In-Meeting Chat

Implemented via Realtime broadcast (no DB persistence by default). Mobile clients should subscribe to meeting-chat-{id} and render messages locally.

```
ts
const chat = supabase.channel(meeting-chat-${meetingCode});
chat.subscribe();
chat.send({ type: 'broadcast', event: 'message', payload: { userName, text, ts: Date.now() } });
```

11. File Sharing

Bucket: meeting-files (private, RLS via table).

Table: meeting_file_shares — meeting_id (text), file_path, file_name, file_type, file_size, uploaded_by, uploaded_at, is_visible.

```
ts
const path = meeting/${meetingCode}/${Date.now()}_${file.name};
```

```

await supabase.storage.from('meeting-files').upload(path, file);
await supabase.from('meeting_file_shares').insert({
  meeting_id: meetingCode, file_path: path,
  file_name: file.name, file_type: file.type, file_size: file.size,
  uploaded_by: userId
});
// Read
const { data } = await supabase.from('meeting_file_shares').select('*').eq('meeting_id', meetingCode);
// Download
const { data: blob } = await supabase.storage.from('meeting-files').download(path);

```

12. Recordings

Bucket: meeting-recordings (private). Table meeting_recordings — host-only RLS.

```

ts
await supabase.storage.from('meeting-recordings').upload(`${meetingCode}/${Date.now()}.webm, blob);
await supabase.from('meeting_recordings').insert({
  meeting_id: meetingCode, host_id: userId,
  file_path, status: 'completed', duration_seconds, file_size,
  ended_at: new Date().toISOString()
});

```

13. Captions

Table: meeting_captions — participants can INSERT (only their own captions); any participant in the meeting can SELECT.

```

ts
await supabase.from('meeting_captions').insert({
  meeting_id: meetingUuid, participant_id: participantRowId, content: transcript
});
supabase.channel(captions-${meetingUuid})
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'meeting_captions', filter: meeting_id=eq.${meetingUuid} },
    ({ new: row }) => render(row))
  .subscribe();

```

Mobile speech-to-text suggestions: iOS SFSpeechRecognizer, Android SpeechRecognizer, or third-party.

14. Notifications & Push

Table: notification_history (user_id, title, message, type, data jsonb, read).

Table: user_push_tokens (user_id, push_token, platform 'ios'|'android').

Register a push token on launch:

```

ts
await supabase.from('user_push_tokens').upsert({ user_id: userId, push_token, platform: 'ios' });

```

15. Recent Meetings & History

Table: user_recent_meetings — full ALL access on own rows.

```

ts
await supabase.from('user_recent_meetings').upsert({
  user_id: userId, meeting_id: code, meeting_title: title, is_host
});
const { data } = await supabase.from('user_recent_meetings')
  .select('*').eq('user_id', userId).order('last_accessed', { ascending: false }).limit(10);

```

16. Activity Logging

Edge function: log-activity (no JWT required).

http

POST /functions/v1/log-activity

```
{ "userId": "...", "action": "meeting_joined", "country": "NG" }
```

Internally calls SQL function log_platform_usage.

17. Storage Buckets Summary

Bucket	Public?	Use
--------	---------	-----

---	---	---
-----	-----	-----

meeting-files	No	Avatars + in-meeting file shares
---------------	----	----------------------------------

meeting-recordings	No	MP4/WebM recordings
--------------------	----	---------------------

Use getPublicUrl only for buckets you make public; otherwise use createSignedUrl(path, 3600) for time-limited mobile playback links.

18. Active Speaker / Video Frame Selection

The web app supports clicking a tile to pin it as the focused frame. Mobile should mirror this:

1. Maintain a local pinnedUserId state.
2. Broadcast on meeting-pin-{id} so co-participants can mirror (optional).
3. Apply audio-energy based active-speaker detection: read RTCPeerConnection.getStats() audioLevel every 500ms, the loudest speaker is auto-promoted to grid position 1 unless a manual pin is set.

19. Color Scheme & Design Tokens

Colors are HSL. Tailwind tokens map to CSS variables in index.css.

Token	HSL	Hex (approx)	Use
-------	-----	--------------	-----

---	---	---	---
-----	-----	-----	-----

--background	240 10% 3.9%	#0a0a0f	Page bg (dark)
--------------	--------------	---------	----------------

--foreground	0 0% 98%	#fafafa	Primary text
--------------	----------	---------	--------------

--card	240 15% 9%	#14141d	Surfaces
--------	------------	---------	----------

--primary	24 100% 50%	#ff6a00	Brand orange (CTAs)
-----------	-------------	---------	---------------------

--primary-glow	24 100% 60%	#ff8533	Hover/glow
----------------	-------------	---------	------------

--primary-accent	16 85% 55%	#e85d2a	Gradient end
------------------	------------	---------	--------------

--secondary	240 100% 50%	#0000ff	Secondary accent
-------------	--------------	---------	------------------

--accent	280 100% 60%	#9933ff	Highlights
----------	--------------	---------	------------

--destructive	0 72.2% 50.6%	#dc2626	End-call, errors
---------------	---------------	---------	------------------

--muted-foreground	240 5% 64.9%	#a1a1aa	Subtle text
--------------------	--------------	---------	-------------

--border	240 15% 12%	—	Hairlines
----------	-------------	---	-----------

--ring	24 100% 50%	—	Focus rings
--------	-------------	---	-------------

Auth screen background gradient (must be reused on mobile): from-purple-900 via-purple-800 to-indigo-900.

Brand gradient: linear-gradient(135deg, hsl(24 100% 50%), hsl(16 85% 55%)).

Typography: Inter (300/400/500/600/700/800/900). Body 16, headings 20–32 with semibold/bold.

Radii: 0.75rem base. Buttons usually rounded-full for the in-call dock.

Shadows: Glow 0 0 40px hsl(24 100% 50% / 0.3); card 0 10px 40px -10px rgba(10,10,15,0.4).

20. Mobile UX Requirements

- Auth screen: split layout — left brand panel (gradient + logo + tagline "Connecting people across the globe"), right form. On phones it stacks; brand panel becomes a 30% header.

- Lobby: full-screen card with looping illustration + "Asking host to let you in..." + cancel button.
- Call screen: paginated video grid (max 9–12 tiles visible). Bottom dock with rounded buttons (Mic, Cam, Captions, Share, Reactions, Hand, Chat, Participants, More, End Call — red).
- Self view: mirrored horizontally (`scaleX(-1)`).
- Tap a tile: pin / unpin.
- Long press a tile (host only): mute that participant.
- Participant sheet: bottom sheet on phones, side drawer on tablets — shows ALL participants, with search + "pin" action.
- Reconnect logic: on app foreground, re-subscribe channels and resume the `RTCPeerConnection` if `iceConnectionState !== 'connected'`.
- Permissions (must request before joining): camera, microphone, notifications, location (optional, for participant city/country tag).

21. Server Keep-Alive

Free-tier Supabase pauses after inactivity. The web app pings `supabase.auth.getSession()` every 4 minutes while open. The mobile app should do the same when active and rely on a server cron (recommended: GitHub Actions hourly curl <https://ytbilvvrqokjqvzbbtpd.supabase.co/auth/v1/health>) for true 24/7 keep-alive.

22. Error Handling Conventions

All Supabase calls return `{ data, error }`. Mobile rules:

- Never throw on error; surface friendly toast.
- Detect `JWT expired!` call `supabase.auth.refreshSession()` then retry once.
- Detect `Network request failed!` queue write, retry with exponential backoff (1s, 2s, 4s)

23. Feature Checklist for the AI

A correct mobile build must include:

1. ' Email/password auth with reset
2. ' Profile edit + avatar upload
3. ' User settings (theme, defaults)
4. ' Dashboard with: instant meeting, join by code, schedule, recent meetings list, sche
5. ' Schedule meeting form + email invitations
6. ' Lobby (knock / admit / deny)
7. ' Video call: WebRTC mesh, mirrored self-view, pinning, paginated grid, "View all"
8. ' Audio-only mode toggle
9. ' In-meeting chat (Realtime)
10. ' File sharing (upload + list + download)
11. ' Captions (live transcription)
12. ' Hand raise + reactions
13. ' Recording (host)
14. ' Push notifications
15. ' Recent meeting history
16. ' Persistence: never drop call on backgrounding; restore on resume.

24. Reference: All Tables Quick Index

profiles, user_settings, user_roles, user_recent_meetings, user_push_tokens, meetings, meeting_participants, meeting_captions, meeting_file_shares, meeting_recordings, meeting_admins, scheduled_meetings, meeting_invitations, notification_history, platform_usage_logs.

All tables are RLS-enabled. Always authenticate the user first; the same anon key is used for both reads and writes — RLS enforces row visibility.